School of Future Tech

Case Study Report

on

Case study 58 :

RageJEX EpicTask Dashboard

by

Yuvraj Jitendra Singh (150096725009)

Student Details
Name:-Yuvraj Jitendra Singh
Roll Number:-150096725009
College:-ITM Skills University,Kharghar
Batch:-2025-2029
Course:BTech (CSE)

Git hub repository link :-
Git Hub live link :-

Index:-

1. Introduction to the Case Study

Project management systems are essential for organizing tasks, monitoring project progress, managing resources, and ensuring timely completion of organizational goals. Traditional project management tools often function only as record-keeping systems and fail to provide intelligent assistance for dependency management, workload balancing, and scheduling optimization.

The RageJEX EpicTask Dashboard is a modern web-based project and task management system developed using React.js and Vite. The application provides an interactive dashboard for managing projects, tasks, team members, dependencies, notifications, and workload distribution. The system integrates advanced algorithms such as Critical Path Analysis, Dependency Cycle Detection, Undo/Redo State Management, FIFO Alert Processing, and Automated Workload Balancing to improve project execution efficiency.

The dashboard is designed as a responsive client-side application that stores data using browser LocalStorage, ensuring fast performance, offline availability, and seamless user interaction.

---

2. Problem Statement / Case Background (Abstract)

Modern software development and project execution involve complex workflows consisting of interconnected tasks, multiple stakeholders, milestones, deadlines, and resource constraints. Existing project management solutions often struggle to provide real-time validation of dependencies, workload distribution, and scheduling risks.

The RageJEX EpicTask Dashboard addresses these challenges by providing an intelligent project management platform capable of:

- Detecting circular task dependencies.
- Calculating project completion timelines using Critical Path Analysis.
- Managing notifications through FIFO queues.
- Supporting Undo/Redo operations through state history tracking.
- Automatically balancing team workloads.
- Validating project milestones through template-based compliance checking.

The application combines algorithmic efficiency with an intuitive user interface, resulting in a highly responsive and productive project management solution.

---

3. Problem Statement / Case Study Design

Problem Statement

Design and develop a React-based single-page application capable of managing projects, tasks, teams, dependencies, and scheduling workflows while ensuring:

- Prevention of circular task dependencies.
- Dynamic project timeline calculation.
- Automated workload distribution.
- Project milestone compliance validation.
- Real-time state tracking and history management.
- Persistent storage using browser LocalStorage.

System Objectives

1. Detect and block circular dependencies in project workflows.
2. Calculate project completion timelines using graph-based algorithms.
3. Balance team workload automatically.
4. Provide Undo and Redo functionality for project modifications.
5. Validate project milestones against predefined templates.
6. Maintain data persistence and synchronization using LocalStorage.

System Modules

- Authentication Module
- Dashboard Module
- Projects Module
- Tasks Module
- Dependencies Module
- Notifications Module
- Workload Management Module
- Settings Module

---

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Technologies Used

Frontend Technologies

- React.js
- Vite
- React Router DOM
- Tailwind CSS

- Lucide React Icons
- Recharts

State Management

- React Context API

Storage

- Browser LocalStorage

Algorithms and Data Structures

1. Critical Path Analysis (DFS + Memoization)

The project tasks are modeled as a Directed Acyclic Graph (DAG). A memoized Depth First Search (DFS) algorithm is used to determine the longest dependency path and calculate the project's total completion duration.

Purpose:

- Identify tasks that directly affect project completion.
- Highlight scheduling bottlenecks.

2. Dependency Cycle Detection (DFS Loop Detection)

Before creating a task dependency, the system validates the dependency graph using DFS traversal and recursion stack tracking.

Purpose:

- Prevent circular references.
- Avoid project deadlocks.

3. Team Workload Balancing Algorithm

A greedy allocation strategy calculates workload scores for each team member.

Formula:

Workload Score =

Active Tasks + Blocked Tasks

The member with the lowest workload score is selected for new task assignments.

4. Undo/Redo State Management

A dual-stack architecture maintains historical project states.

Components:

- Past Stack
- Current State
- Future Stack

This enables users to revert and reapply project changes efficiently.

5. FIFO Alert Queue

Notifications are managed using a First-In First-Out queue structure.

Purpose:

- Organize alerts sequentially.
- Maintain notification history.

---

5. Problem Statement / Case Study Implementation Details and Snapshots

System Architecture

The system follows a client-side architecture consisting of:

- React Router for navigation.
- Context API for centralized state management.
- Validation layer for dependency checking.
- Recharts visualization engine.
- Undo/Redo history manager.
- LocalStorage persistence layer.

Key Functional Features

Authentication System

- User Login
- User Registration
- Password Validation
- Protected Routes

Project Management

- Create, Update, Delete Projects
- Project Tree View
- Folder-based Project Organization

Task Management

- Task Creation
- Priority Management
- Deadline Tracking
- Dependency Management

Advanced Features

Template Checker & Milestone Validator

- Software Release Templates
- Marketing Campaign Templates
- Product Launch Templates
- Compliance Verification

Deadline Risk Sorter

- Delay Analysis
- Risk Score Calculation
- Priority-based Sorting

Task Lock Hub

- Blocked Task Detection
- Dependency Visualization

Quickest Project Completion Finder

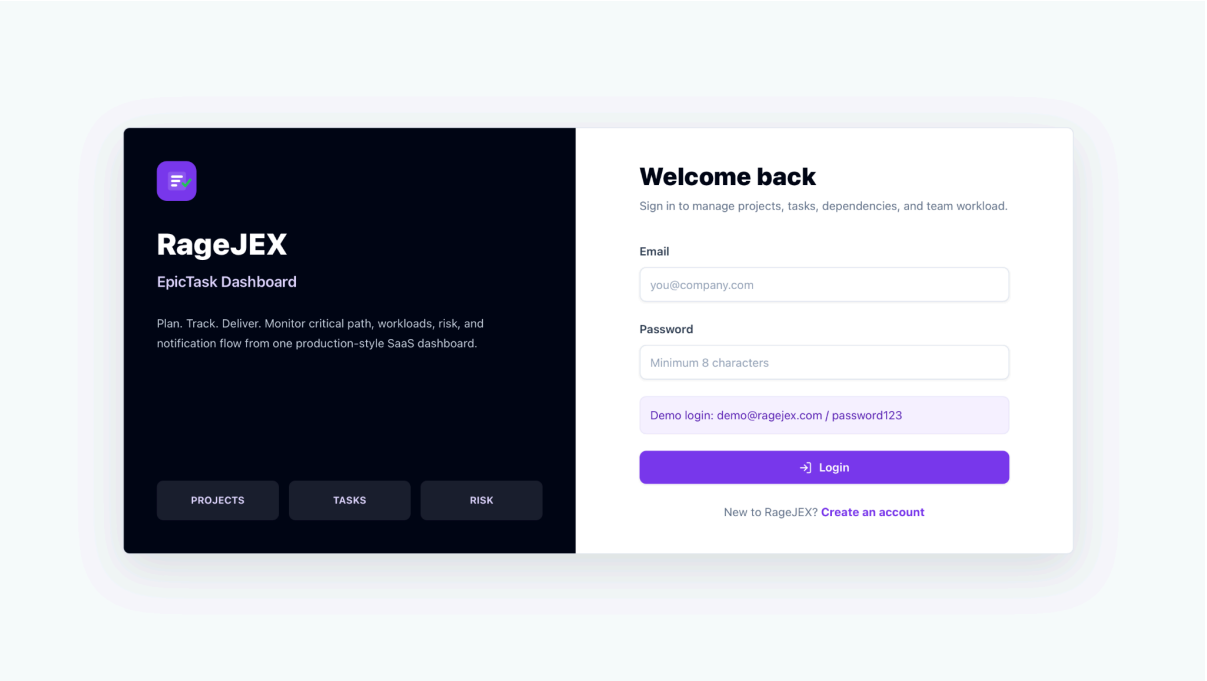
- Critical Path Visualization
- Timeline Optimization

Team Workload Assignor

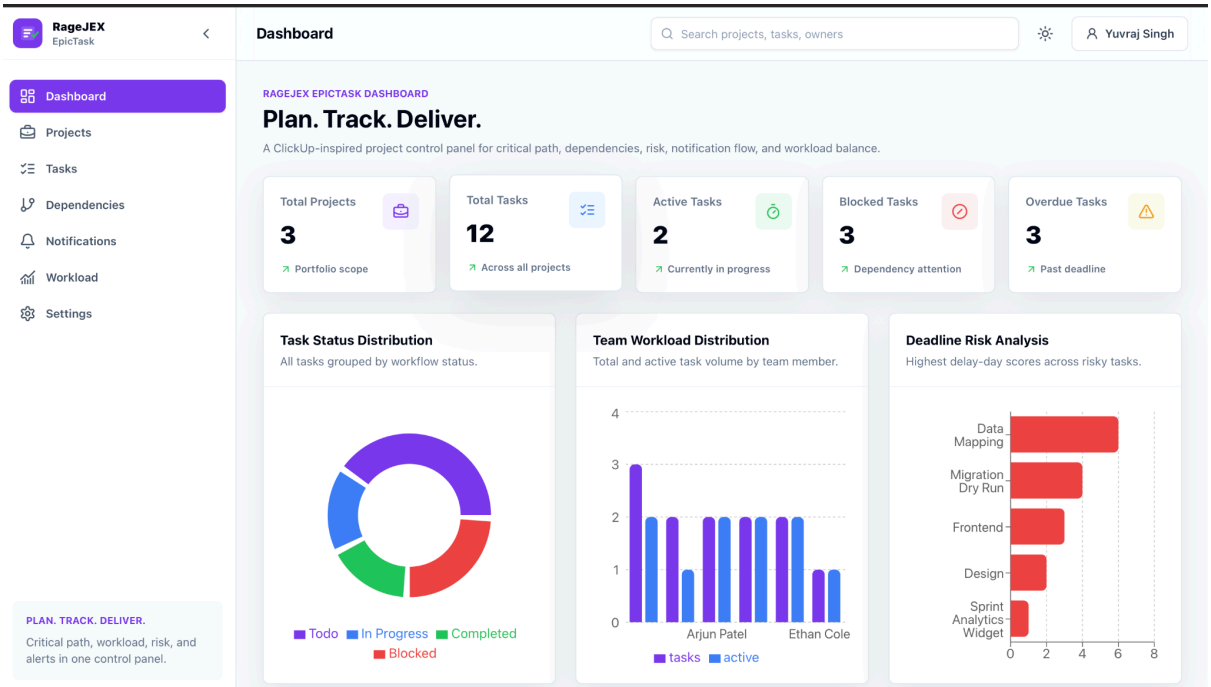
- Automatic Resource Allocation
- Workload Distribution Charts

Snapshots :-

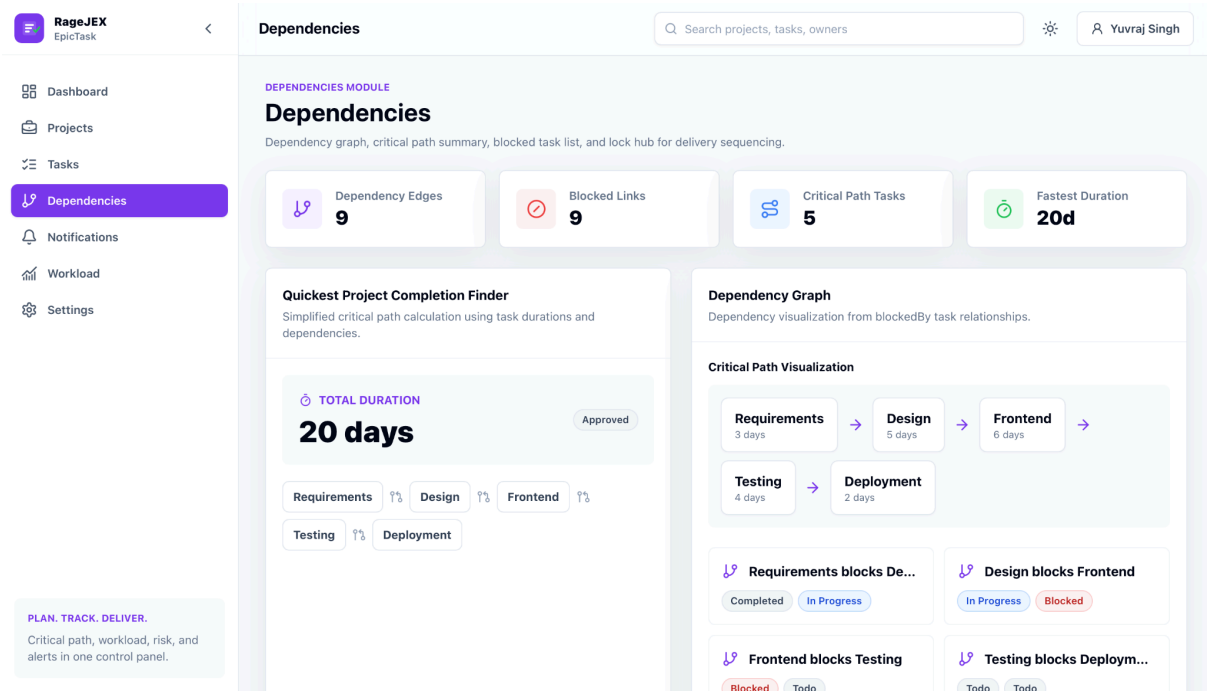
1. Signup Page



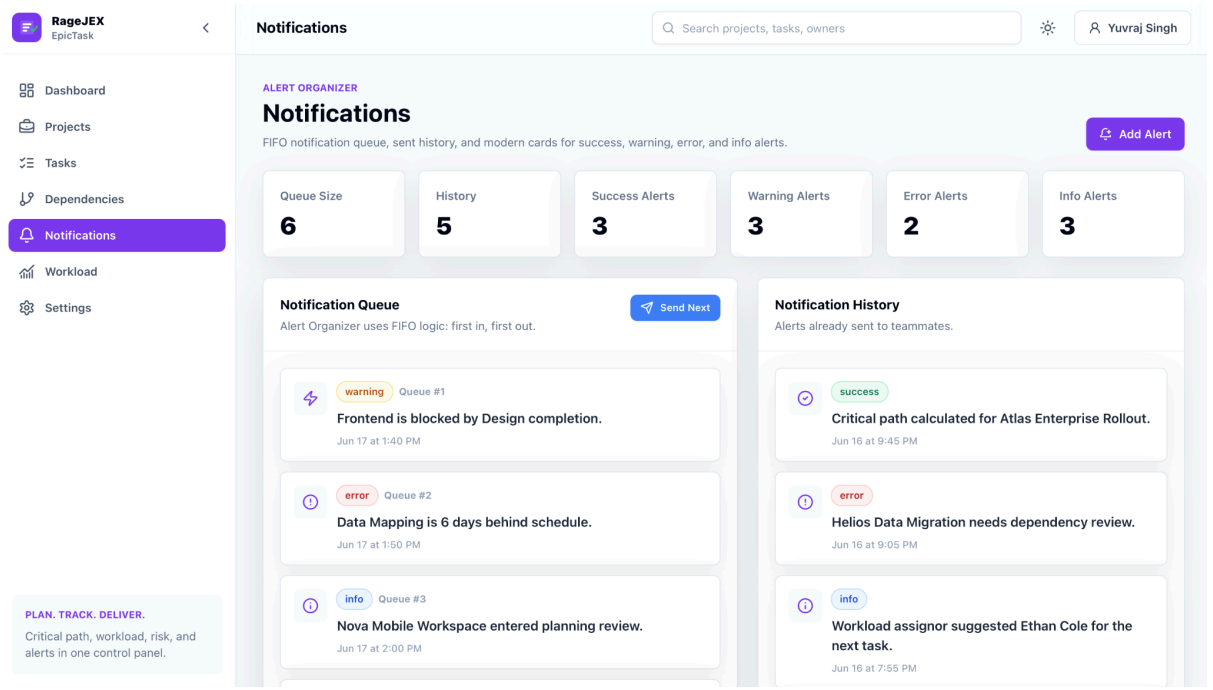
2. Dashboard Overview



3. Dependency Visualizer



4. Notification Queue Screen



6. Problem Statement / Case Study Results and Conclusion

Results

The developed system successfully achieved all intended objectives.

Key outcomes include:

- Successful implementation of dependency cycle detection.
- Accurate critical path calculation for project scheduling.
- Automated workload balancing across team members.
- Functional Undo/Redo state management.
- Template-based milestone validation.
- Responsive user interface across desktop and mobile devices.
- Efficient LocalStorage-based persistence mechanism.

Testing and Validation

Build Testing

- Successful Vite production build.
- 2266 modules compiled successfully.

Functional Testing

- Authentication validation verified.
- Dependency cycle detection tested successfully.
- Workload balancing calculations validated.

Responsive Testing

- Tested on mobile (360px) to desktop (1440px) viewports.

Conclusion

The RageJEX EpicTask Dashboard successfully demonstrates the integration of advanced algorithms within a modern web-based project management platform. By combining graph theory, greedy algorithms, queue structures, and state management techniques, the system actively assists users in project planning and execution.

The application provides efficient project monitoring, workload optimization, dependency validation, and scheduling support while maintaining high responsiveness and offline accessibility. The project serves as a strong foundation for future

enterprise-level project management systems and highlights the practical application of algorithms and data structures in modern frontend development.

---

7. References

1. React Documentation – https://react.dev
2. Vite Documentation – https://vite.dev
3. Tailwind CSS Documentation – https://tailwindcss.com
4. MDN Web Storage API Documentation – https://developer.mozilla.org
5. Cormen, Thomas H., et al. *Introduction to Algorithms*. MIT Press.
6. Recharts Documentation – https://recharts.org
7. React Router Documentation – https://reactrouter.com